Quantifying Augmentation Leakage in Soil Image Classification: A Source-Disjoint Evaluation within a Low-Cost Edge IoT Irrigation System

Utkarsh Persai[1], Yash Chaubey[1], Venkata Hrishi Samay Jetty[1], Devansh Rathore[1], Syamasudha Veeragandham[1,*]

[1]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014

*Corresponding author: syamasudha.v@vit.ac.in

Abstract

Soil photographs are increasingly classified with convolutional neural networks, and the accuracies published for this task usually exceed 90%. When the dataset behind such a figure is built by cutting a few dozen field photographs into tiles and augmenting them before the split, near-copies of one photograph end up on both sides of it. We measured how much this matters. The soil classifiers of a low-cost (₹970) edge irrigation node, built on an ESP32 with a DHT11, a capacitive moisture probe and a relay-driven pump, were evaluated twice on identical features: once by stratified cross-validation over augmented samples, and once with every tile of a held-out photograph excluded from training. Across three frozen ImageNet backbones and one fixed RBF support vector machine, five-class soil type fell from 91.9–94.0% to 44.6–46.8%, close to its 43.3% majority-class baseline, and four-class soil condition fell from 98.6–99.7% to 75.5–86.1%. Over five tiling seeds the soil-type gap was 48.1 ± 1.3 percentage points, and an exact permutation test over photographs gave $p = 3.1 \times 10^{-5}$ for every backbone; the smaller condition gap rested on a few photographs and was significant for two backbones of three. Two findings explain the collapse: the same features identify which photograph a tile came from with 91.5% accuracy, and classes represented by a single photograph score an F1 of zero once it is held out. On the hardware, loss of pump power went undetected while the firmware kept reporting irrigation. A Monte Carlo check of a proposed pulse-response test, calibrated on normal behaviour only, flagged every simulated fouled, dislodged, disconnected or submerged probe. We recommend that soil-image accuracy be reported only with a photograph-level split and a per-class count of source photographs.

Keywords: Precision agriculture; Soil image classification; Data leakage; Grouped cross-validation; Transfer learning; ESP32; Internet of Things; Smart irrigation

1. Introduction

Irrigation is the largest single use of developed freshwater, and the room for waste is shrinking. Morchid et al. summarise projections of a gap of roughly 40% between freshwater supply and demand by 2030 [1]. Against that background, irrigating according to what the soil actually needs, rather than on a timer, is one of the more direct ways agriculture can reduce its draw on water.

India illustrates why cost matters as much as capability. It is among the most active countries in IoT irrigation research [2], yet most of its farms are smallholdings where equipment has to be cheap enough to buy outright and simple enough to install without a technician. A system that is accurate in a laboratory but costs several thousand rupees, or that stops working when the mobile network drops, will not be adopted there.

The typical IoT irrigation node is now well understood. A systematic review of 92 studies from 2014 to 2024 found that most use an Arduino, Raspberry Pi, ESP8266 or ESP32, communicate over WiFi,

LoRa or ZigBee, and are built for remote monitoring, water saving and soil moisture sensing [2]. The hardware we describe follows this pattern closely, and we do not claim novelty for it.

What most of these nodes share is an open-loop control rule. A moisture reading is compared with a fixed threshold and the pump is switched on or off [3]. The controller never checks whether the water it commanded actually arrived, so compacted soil, surface runoff or a failed pump all look the same to it as a successful irrigation.

They also trust the moisture probe without question. Capacitive probes lose accuracy in the field through mineral deposits on the electrodes, corrosion, temperature effects on the dielectric and gradual loss of contact with the soil. A recent review of moisture sensing lists temperature drift, the need for site-specific calibration and poor reproducibility in mixed soils among the open problems [4]. The difficulty is that a degraded probe does not fail visibly; it keeps reporting numbers in a normal range.

Work on this problem has focused on calibration. A deep-learning method recalibrates a probe in place from two reference points, saturation and field capacity [5]. A semi-automatic rig with curve-fitted models reduces the manual effort involved [6], and an automated two-step correction can be applied retrospectively to logged network data [7]. Each of these either works after the fact, needs a person, or waits for the soil to reach a known state. None tells the controller, before its next actuation, whether the probe can be trusted.

Cameras have been added to these systems to classify the soil itself. Transfer learning from ImageNet-pretrained networks is the usual approach: a MobileNetV2-based model has been reported for soil type [8], and a comparison of five backbones on 1,269 images found ResNet50 best at 97.3%, with EfficientNet-B0 at about 91% offering the best trade-off between speed and accuracy [9]. More recent systems combine image features with measured soil properties [10], and surveys describe multimodal fusion as a mature line of work [11].

The accuracy of such classifiers depends on how they are evaluated, and this is where a well-documented problem enters. Kapoor and Narayanan found data leakage in 294 papers across 17 fields [12]. In medical imaging, splitting per image instead of per subject inflated accuracy by about 5 to 30 percentage points [13], and in digital pathology, tiles cut from the same slide on both sides of a split produced the same kind of error [14]. The mechanism is simple: when near-copies of a sample appear in training and test data, the model is partly tested on what it has already seen.

The remedy is not new either. Grouped or block cross-validation, in which related samples are kept together, has been standard advice in ecology for years [15]. In soil science it has recently been argued for three-dimensional soil mapping, where depth samples from one profile must not be split [16], and for tabular soil data, where holding out whole soils reduced the R^2 of some models from 0.62 to below zero [17]. We therefore do not present source-level splitting as a contribution; we apply it to a setting where its effect has not been measured.

Soil photographs are more exposed to this problem than most image data, for three reasons. First, a soil photograph is a texture with no object in it, so the standard way to obtain more training images is to cut many crops from one high-resolution photograph; the pipeline studied here takes 37 crops and resized views from each. In leaf-disease datasets each leaf is usually photographed once, which is why runtime augmentation after the split is enough to prevent leakage there [18]. Second, each soil photograph normally corresponds to one field sample, and a class may be represented by only one or two samples, so holding out a photograph can remove a class entirely. Third, every crop of a photograph shares its lighting, white balance, camera and moisture state, which gives the classifier an easier cue than soil texture to learn. The largest recent field study we found drew on about 10,000 field images from 21

counties [19]; with a source base of that size the problem is far less severe, although its abstract does not say whether patches cut from one image were kept on one side of the split.

To check whether this gap had been measured, we ran a keyword search in September 2026 combining soil image classification with data leakage, grouped cross-validation and image-level splitting, and confirmed each candidate through its publisher record. The search returned the leakage studies on tabular soil data [16], [17], the pathology work [14] and a large field-image study [19]. It did not return a study that evaluates the same soil-image model under both protocols and reports the difference. To our knowledge, therefore, the measurement reported here is the first of its kind for soil photographs.

Deployment is the last consideration. Many IoT irrigation systems send data to a cloud service for decisions; the baseline system for this work depends on a ThingsBoard instance [1]. Edge-first designs that keep decisions local are increasingly recommended for farming [20], and in rural India the reason is practical: connectivity is intermittent, and an irrigation controller that stops when the network drops is not usable.

The paper makes four contributions. (i) It measures the gap between augmentation-level and source-disjoint evaluation for soil type and soil condition classification, across three backbones and five tiling seeds, with photograph-level confidence intervals and an exact permutation test. (ii) It identifies why the gap is so large for soil photographs, using a photograph-identification test and a per-class analysis of source counts. (iii) It reports a costed, hardware-tested edge irrigation node and describes the one fault it cannot detect. (iv) It checks, in simulation, whether a proposed pulse-response test would detect that fault and several probe failures, and states where the test stops working.

1.1 Research Objectives

1. To measure the difference between augmentation-level and source-disjoint cross-validation for CNN-feature soil classifiers, with uncertainty estimates that treat the source photograph as the unit of independence.
2. To test whether that difference depends on the choice of backbone or on the random tiling, and to explain it in terms of properties of soil image datasets.
3. To build, cost and test an edge-first irrigation node that keeps working without a network, and to establish which of its faults are detectable.
4. To assess, in a simulation grounded in measured sensor noise, whether a pulse-response test can detect the faults the current firmware cannot.

2. Related Work

Morchid et al. describe the irrigation system used as our baseline [1]. An ESP32 reads temperature, air humidity, soil moisture and tank water level, sends them to ThingsBoard, and switches a pump by threshold. The authors report lower water use at a cost of about US$44. The system has no image input, depends on the cloud for decisions, and has no way of checking its sensors.

The review by Abdelmoneim et al. is the most thorough map of IoT irrigation research [2]. It combines a bibliometric analysis of 290 Web of Science records with a PRISMA-guided review of 92 studies. For our purposes its most telling result is an absence: across those 92 studies there is no sustained treatment of probe drift, sensor validation or data quality.

Sreeram et al. reach the same problem from the sensing side [4]. Their review of moisture sensing technologies identifies temperature drift, site-specific calibration and poor reproducibility in

heterogeneous soils as unsolved. The problem is therefore recognised by those who build sensors but rarely handled by those who build irrigation controllers.

Calibration research improves the accuracy of a working probe but does not decide whether the probe is working. The two-step correction of Gasch et al. is applied to logged data [7]; Pahuja's recalibration system still needs an operator and a rig [6]; and the self-calibration method of Aranda Britez et al., the most advanced of the three, needs the soil to pass through saturation and field capacity and cannot tell a drifted probe from a failed one [5].

Sensor fault detection has been handled at network level. Yang et al. flag a faulty node by comparing it with a weighted average of its neighbours and with its own history, which catches communication errors, flat batteries and electronic failures [21]. The method needs neighbours, which a single-probe node does not have, and a history-based check will not catch slow drift, because the drift becomes part of the history.

For soil images, Banoth and Ramana Murthy report a MobileNetV2 model with a custom head and very high per-class accuracy [8], and Jalapur and Patil compare five backbones on 1,269 images [9]. Neither describes the split at the level of source photographs, and neither links classification to actuation. Their datasets differ from ours, so we compare protocols rather than numbers. Qin et al. classified residual slope soils from about 10,000 field images collected in 21 counties, cut into 3,380 soil patches and augmented thirteen-fold, and report 91.5% on an independent test set of 6,591 images [19]. Their source base is more than 600 times larger than ours; the abstract does not state whether patches from one field image were kept together.

AgroSense is the published system closest to ours in structure [10]. It combines soil image features with nutrient measurements for crop recommendation and shows that images add information that tabular data lack. Because it stops at a recommendation, an unreliable input does not lead to a physical action, which is the situation in an irrigation controller.

Leakage in machine learning has been documented widely. Kapoor and Narayanan catalogue eight kinds of leakage across 17 fields [12]. Tampu et al. show the effect of per-image splitting on retinal images [13], and Bussola et al. show that tiles from one slide on both sides of a split inflate results in pathology [14], the closest analogue to soil tiling. Tools for detecting duplicated or near-duplicate images in large geospatial collections have been built for the same reason [22], and plant-disease work has moved augmentation after the split to avoid the problem [18].

Grouped validation is established in the environmental sciences. Roberts et al. showed that random cross-validation underestimates error whenever data have spatial, temporal or hierarchical structure, and recommended block designs [15]. John et al. showed that three-dimensional soil maps need leave-profile-out validation [16], and Chen and Vanderborght showed that soil-wise splitting sharply lowers the apparent skill of models trained on tabular soil data [17]. None of these studies concerns soil photographs.

Finally, edge deployment has been argued on operational grounds. Tariq et al. describe an edge framework that runs lightweight deep learning on IoT data without cloud dependence [20]. We take the same position for a narrower reason: the irrigation loop must keep running when the network does not.

Table 1: Representative prior work and its relationship to this study.

Prior work	Focus	Method / platform	Relationship to this work
Morchid et al. [1]	IoT irrigation for water saving	ESP32, four sensors, ThingsBoard cloud, fixed threshold, ≈US$44	Baseline. We keep the sensing concept, move decisions to the edge and add an image channel.

Prior work	Focus	Method / platform	Relationship to this work
Abdelmoneim et al. [2]	Review of IoT irrigation, 2014–2024	PRISMA review of 92 studies	Shows probe drift and data quality are largely unaddressed in the field.
Aranda Britez et al. [5]	In-situ probe recalibration	Deep model anchored on saturation and field capacity	Closest calibration method; passive, and gives no fault verdict.
Jalapur & Patil [9]; Banoth & Ramana Murthy [8]	CNN soil type recognition	Transfer learning; up to five backbones	Define the reported accuracy band; split protocol not stated at source level.
Qin et al. [19]	Field images of residual slope soils	ResNet18; ≈10,000 field images, 3,380 patches, 13× augmentation	Large source base; grouping of patches by source image not stated in the abstract.
Tampu et al. [13]; Bussola et al. [14]	Leakage in medical and pathology imaging	Per-image vs per-patient; tile vs slide splits	Closest methodological analogues; we measure the same effect for soil photographs.
Roberts et al. [15]; John et al. [16]; Chen & Vanderborght [17]	Grouped validation in ecology and soil science	Block, profile-wise and soil-wise cross-validation	Establish the remedy for structured, non-image data.

Table 1 places this study against the work it draws on. The baseline system [1] shares the sensing concept but keeps its decisions in the cloud, while the leakage studies [13], [14] supply the method rather than the domain. No row combines a soil-image classifier with a stated photograph-level split, which is the gap this paper measures.

2.1 Gaps

- G1. The soil-image studies reviewed here do not state whether tiles or augmented copies of one photograph were kept on one side of the split [8], [9], [19], and the size of the resulting error has not been measured for soil photographs.
- G2. Threshold irrigation controllers do not verify that a commanded action had any effect [1], [3].
- G3. Controllers accept the probe reading as true; the review of 92 studies in [2] finds no systematic treatment of this.
- G4. Existing fault detection needs neighbouring sensors [21], and existing calibration is retrospective, manual or passive [5]–[7].

2.2 Problem Statement

CNN soil classifiers are reported at more than 90% accuracy and are starting to drive physical actions in irrigation systems. Soil image datasets can be small in terms of field photographs and large in terms of tiles, as IRSID is, and the studies reviewed here do not state whether tiles from one photograph were kept together when the data were split. It is therefore unclear how well such classifiers recognise soil they have not seen. The controllers they feed, in turn, act on a single probe without checking either the probe or the result of the action. This study addresses both halves: it measures what the classifiers achieve on unseen photographs and why, and it tests the controller against physical faults.

3. Materials and Methods

3.1 System architecture

The system runs entirely on local equipment in four layers. The ESP32 reads the sensors and drives the relay and status LEDs. Newline-delimited JSON is sent over USB serial at 115200 baud to a laptop, where a Python collector parses and stores it. A Streamlit dashboard shows live values, history and

model outputs in a browser, and offers an advisory watering schedule that combines the classified soil condition with a 48-hour forecast from the Open-Meteo API [23]. The advisory is informational; the relay is controlled only by the firmware, so irrigation continues if the laptop or the network is unavailable (Fig. 1).

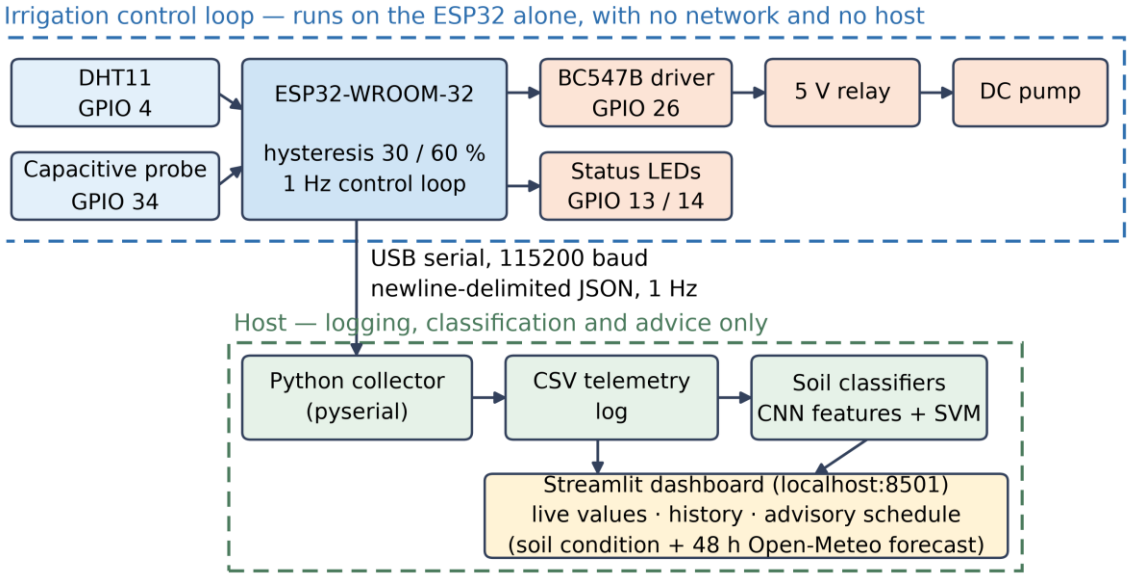


Figure 1: System architecture and data path. Irrigation control runs on the ESP32; the host adds logging, classification and an advisory schedule.

Figure 1 shows the four layers and, more importantly, where the boundary between them falls. The ESP32 reads both sensors, applies the hysteresis rule and drives the relay by itself once a second; everything past the serial link logs, classifies and advises. Because no host process can actuate the relay, irrigation continues unchanged when the laptop or the network is unavailable.

3.2 Hardware, circuit and cost

The node uses an ESP32-WROOM-32 development board. A DHT11 on GPIO 4 measures air temperature and humidity, and a capacitive soil moisture sensor (v1.2) feeds ADC1 channel 6 on GPIO 34. The ESP32 pin cannot drive a relay coil directly, so GPIO 26 switches a BC547B transistor through a 1 kΩ resistor, and the transistor drives a 5 V relay module that switches a small DC pump on a separate supply. Two LEDs on GPIO 13 and 14 show that the system is alive and that the pump is on. The pump supply and the ESP32 share a ground. Early prototyping used a DHT22; the build tested here uses the cheaper DHT11, and all results refer to it. Figs. 2 and 3 show the circuit and the assembled node, and Table 2 lists the pins.

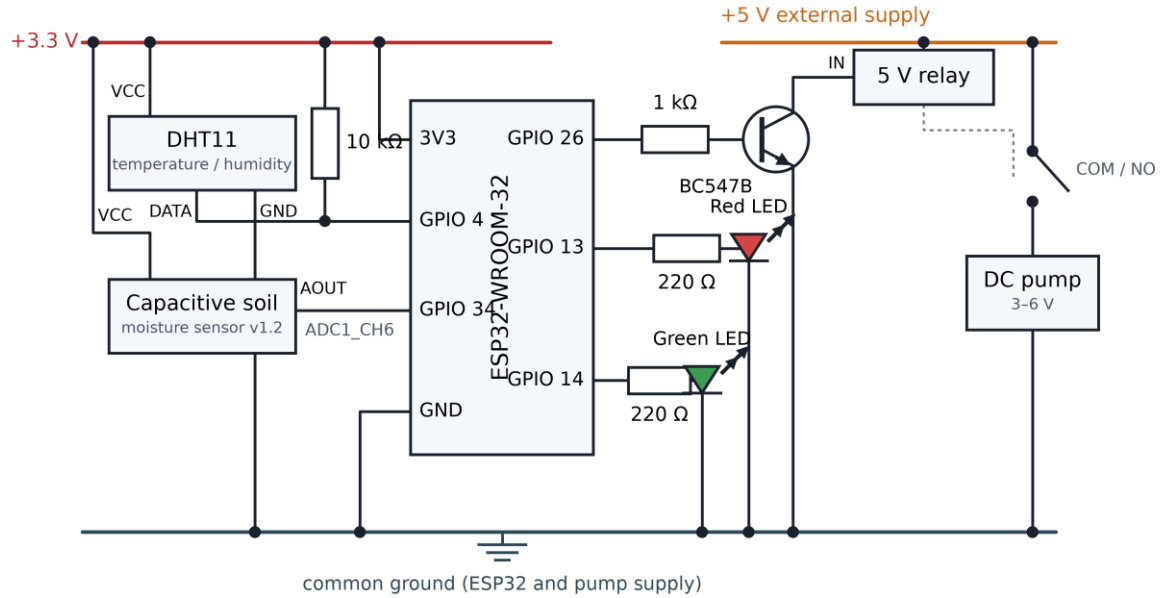


Figure 2: Circuit schematic of the node: DHT11, capacitive moisture sensor, BC547B driver, relay and status LEDs.

Figure 2 gives the wiring behind Table 2. Both sensors sit on the 3.3 V rail, GPIO 26 drives the relay through a BC547B and a 1 kΩ base resistor, and the two status LEDs are current-limited at 220 Ω. The pump runs from a separate 5 V supply that shares a ground with the ESP32, so the relay contact switches the pump circuit alone.

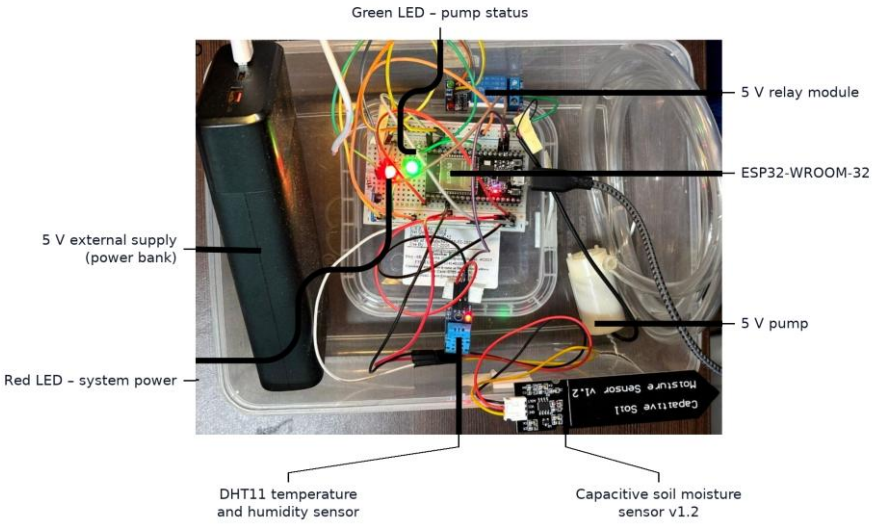


Figure 3: Assembled prototype. The soil is held in a sealed container for controlled wetting and drying.

Figure 3 shows the assembled node. The probe sits in a sealed container of soil that could be wetted and left to dry, which is how every hardware test and the whole telemetry log were produced. The arrangement is repeatable and convenient, but it is not a field deployment, a limitation returned to in §5.

Table 2: ESP32 pin assignments.

Pin	Component	Connection	Function
GPIO 4	DHT11 data	Digital, 10 kΩ pull-up to 3.3 V	Air temperature and humidity

Pin	Component	Connection	Function
GPIO 34	Capacitive moisture sensor	Analogue input, ADC1_CH6	Soil moisture
GPIO 26	BC547B base via 1 kΩ	Digital output	Relay drive
GPIO 13	Red LED via 220 Ω	Digital output	System alive
GPIO 14	Green LED via 220 Ω	Digital output	Pump on (blinks every 250 ms)
3.3 V / GND	Sensors; all components	Supply; common ground	Ground shared with the pump supply

Table 2 lists the five GPIO assignments the firmware uses: GPIO 4 for the DHT11, GPIO 34 for the analogue moisture channel, GPIO 26 for the relay driver, and GPIO 13 and 14 for the status LEDs. GPIO 34 is input-only and carries ADC1_CH6, which is why the probe is read there. Every measurement and hardware test reported below was taken on this wiring.

Table 3 itemises the cost. Four modules are priced from a single Indian retailer's listings [24]; the remaining small parts are shown as the balance of the purchase total the team recorded, rather than as individually verified prices.

Table 3: Bill of materials.

Item	Qty	Price (₹)	Basis
ESP32 development board (ESP-WROOM-32)	1	284	Listed price, RKI-1454 [24]
Capacitive soil moisture sensor v1.2	1	83	Listed price, RKI-3225 [24]
DHT11 temperature and humidity module	1	61	Listed price, RKI-3154 [24]
5 V single-channel relay module	1	42	Listed price, RKI-3151 [24]
Subtotal, itemised modules		470	
DC mini pump (3–6 V), BC547B, two LEDs, 1 kΩ and 2 × 220 Ω resistors, breadboard, jumper wires, tubing	—	500	Balance of recorded purchase total
Total		970	≈ US$11–12
External 5 V supply (power bank)	1	—	Existing equipment, not included
Optional: ACS712 current sensor (§4.3)	1	*145*	Listed price, RKI-1838 [24]

Table 3 shows that four purchased modules account for ₹470 of the ₹970 total, the balance covering the pump, transistor, LEDs, resistors and consumables. At roughly US$11–12 the node costs about a quarter of the US$44 baseline [1]. The optional ACS712 current sensor at ₹145 would add about 15% to the total and is the cheapest route to detecting the fault discussed in §4.3.

3.3 Firmware

The raw ADC value r is converted to a moisture percentage M with a two-point calibration, using the reading in air, A = 2590, and in water, W = 1500:

$$M = clamp(\,100 \cdot (r - A) / (W - A),\ 0,\ 100\,) \qquad (1)$$

where:

M: soil moisture reported by the firmware, % (0–100);

r: raw ADC reading on GPIO 34, dimensionless (0–4095, 12-bit);

A: reading with the probe in dry air, 2590 (firmware MOISTURE_AIR);

W: reading with the probe in water, 1500 (firmware MOISTURE_WATER);

clamp(x, 0, 100): limits x to 0–100, implemented as Arduino constrain(); the mapping itself uses integer arithmetic.

The clamp matters in practice. When the probe was disconnected in test TC-06, the ADC read full scale and the clamp returned 0% instead of a large negative number. GPIO 34 has no internal pull-up, so the value of a floating input is not guaranteed; full scale is what we observed, not something the design relies on.

The pump follows a hysteresis rule with a dry threshold T_D = 30% and a wet threshold T_W = 60%. With u_t the pump state at sample t,

$$u_t = 1 \text{ if } M_t < T_D;\quad u_t = 0 \text{ if } M_t > T_W;\quad u_t = u_\{t-1\} \text{ otherwise} \qquad (2)$$

so that inside the band the previous state is kept and small fluctuations cannot toggle the relay. Each loop reads the sensors, applies Eq. (2), updates the LEDs, prints one JSON record and waits 1 s. If the DHT11 read fails, temperature and humidity are sent as null and irrigation continues on moisture alone. Fig. 4 shows the loop.

where:

u_t: pump state at sample t (0 = off, 1 = on);

M_t: moisture at sample t, % (0–100);

T_D: dry threshold, 30% (firmware MOISTURE_DRY);

T_W: wet threshold, 60% (firmware MOISTURE_WET);

t: sample index, one per second (delay(1000) in the loop).

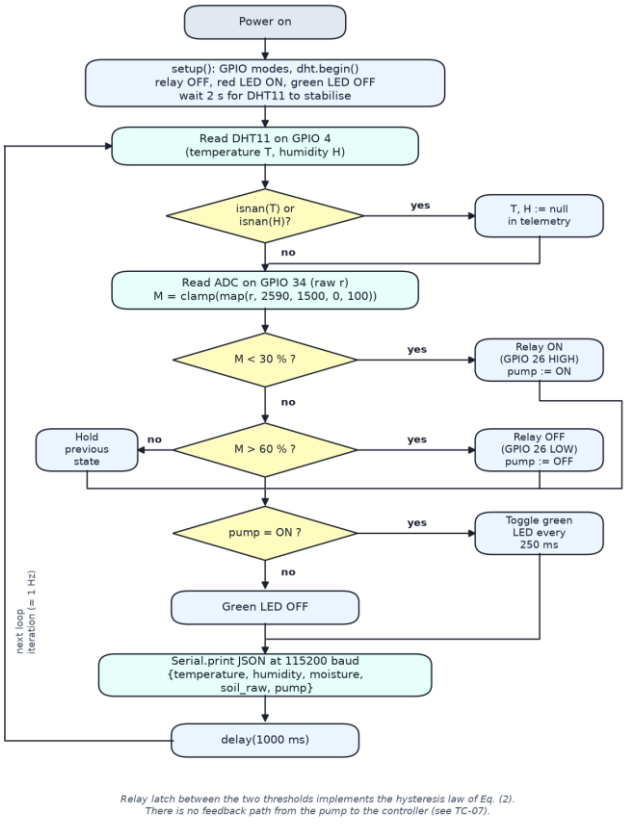


Figure 4: Flowchart of the implemented firmware loop.

Figure 4 traces one pass of the implemented loop. Each iteration reads both sensors, applies Eq. (2), updates the LEDs, prints one JSON record and waits a second; a failed DHT11 read sends nulls and irrigation continues on moisture alone. The figure also shows what is absent: no path returns from the pump to the controller, which is the structural cause of TC-07.

3.4 Datasets

Table 4: Datasets used.

Dataset	Source	Composition	Labels	Use
Soil type	IRSID [25]	16 source photographs. From each: 20 random crops, 15 multi-scale crops, a centre crop, a full resize and 7 rotated versions (0–180°) = 44 images, then five augmented copies of each with the original kept = 264 per source (234 for one smaller photograph). Total 4,194.	Clay, Loam, Loamy sand, Sand, Sandy Loam	Classifier evaluation
Soil condition	Web image search, curated by the authors	14 source images; 20 augmented copies of each with the original kept = 21 per source. Total 294.	Healthy, Cracked, Waterlogged, Compacted	Classifier evaluation
Telemetry	ESP32 node	8,171 records in 33 logging sessions between 5 Feb and 2 Sep 2026, 4.5 h in total; median interval 1.00 s. The longest session is 44.5 min.	Moisture, temperature, humidity	Control-law and noise characterisation

Table 4 sets out the three datasets. The soil-type set expands 16 source photographs into 4,194 samples and the condition set expands 14 into 294, so both are large in samples and very small in photographs — the condition under which the choice of split protocol matters most. The telemetry log contributes 8,171 records over 4.5 h and is used only for control-law and noise characterisation.

The number of source photographs per class is very uneven, and this turns out to matter. For soil type the counts are Clay 7, Sand 5, Loamy sand 2, Loam 1 and Sandy Loam 1; for soil condition they are Healthy 4, Cracked 4, Waterlogged 3 and Compacted 3. The IRSID labels come from particle-size analysis of each sample. The soil-condition images were gathered through a web image search and screened by hand; they are not our own field photographs, and the provenance of individual images was not recorded. The telemetry log stores the mapped percentage but not the raw ADC value, so readings at 0% cannot be told apart afterwards from genuinely air-dry soil; the session analysed in §4.4 was chosen partly because it starts well above that floor.

3.5 Features and classifier

Each 224 × 224 image is passed through a frozen ImageNet-pretrained backbone [26] with global average pooling. We use three backbones of different design and size: EfficientNetV2-B0 [27] and MobileNetV2 [28], each giving 1,280 features, and ResNet50 [29], giving 2,048. For soil type the embedding φ(I) is joined to a 102-dimensional colour descriptor c(I), made of a 32-bin histogram per RGB channel plus the mean and standard deviation of each channel, as in the deployed pipeline:

$$x = [\, \varphi(I) \,,\, c(I) \,] \qquad (3)$$

where:

I: input RGB image, 224 × 224 × 3, pixel values 0–255;

φ(I): frozen backbone embedding after global average pooling; 1,280 values for EfficientNetV2-B0 and MobileNetV2, 2,048 for ResNet50;

c(I): colour descriptor, 102 values — a 32-bin histogram for each RGB channel, each histogram normalised to sum to 1 (each bin 0–1), plus the mean and standard deviation of each channel divided by 255 (0–1); used for soil type only;

x: concatenated feature vector, 1,382 values for the two 1,280-dimensional backbones and 2,150 for ResNet50;

[· , ·]: concatenation.

The classifier is a support vector machine with a radial basis function kernel [30], C = 10, γ = scale and class weights inversely proportional to class frequency. These values were fixed before any evaluation and never tuned, so hyperparameter selection cannot leak test information. For multiple classes, scikit-learn trains one classifier per pair of classes and takes a vote; the pairwise decision for classes k and l is

$$f_kl(x) = sign(\Sigma_i\, \alpha_i\, y_i\, exp(-\gamma\, \|x - x_i\|^2) + b_kl) \qquad (4)$$

where:

f_kl(x): decision for the pair of classes k and l; scikit-learn SVC votes over all K(K − 1)/2 pairs, with K = 5 for soil type and K = 4 for soil condition;

x_i: support vectors drawn from the training folds;

y_i: pairwise labels, ±1;

α_i: learned dual coefficients, $|\alpha_i| \leq C$ with C = 10 (fixed a priori);

γ: RBF kernel width, gamma='scale' = 1/(n_features · Var(X)) computed on each standardised training fold (> 0);

b_kl: bias for that pair;

‖·‖: Euclidean norm.

Features are standardised with a scaler fitted inside each training fold. Fitting the scaler on all data before cross-validation, as the original training script did, is itself a small leak, and we avoid it here. The same classifier, features and settings are used for both protocols, so the only thing that changes between the two reported accuracies is how the data are split. The deployed soil-condition model has a trained dense head; it is not used for any figure in this paper, so that both tasks are evaluated in the same way.

3.6 Evaluation protocols

The first protocol, which we call augmentation-level, is the usual one: the data are expanded first and stratified 5-fold cross-validation is run over the expanded set, so tiles and copies of one photograph are spread across folds. The second, source-disjoint, labels each sample with the photograph it came from and holds out one photograph at a time, so a test photograph contributes nothing to the model that is tested on it (Algorithm 1). The second protocol trains on 93–94% of the data in each fold against 80% for the first, so it has more training data available; any drop it shows is therefore not caused by a smaller training set.

```
Algorithm 1: Source-disjoint evaluation
Input : photographs I_1..I_n with labels y_1..y_n; expansion procedure E
Output: accuracy on unseen photographs
1  for i = 1..n:
2      X_i <- features( E(I_i) )          // tiles, rotations, augmented copies
3      g_i <- i                           // group = source photograph
4  X <- concat(X_1..X_n);  g <- concat(g_1..g_n)
5  for each photograph h in 1..n:
6      fit scaler and SVM on { X[j] : g[j] != h }
7      predict           on { X[j] : g[j] == h }
8  return accuracy over all predictions
```

3.7 Statistical analysis

Accuracy, per-class precision, recall and F1 are reported, with the protocol gap defined as

$$\Delta = Acc_aug - Acc_disjoint \qquad (5)$$

where:

Δ: protocol gap, percentage points (−100 to +100; positive means the augmentation-level protocol scores higher);

Acc_aug: accuracy under stratified 5-fold cross-validation over the augmented samples, % (0–100);

Acc_disjoint: accuracy under leave-one-photograph-out cross-validation, % (0–100).

Because the tiles of one photograph are not independent, uncertainty is estimated at the photograph level. Confidence intervals come from a bootstrap that resamples source photographs with replacement 4,000 times [31]. For comparison we also give the Wilson interval [32] computed as if every sample were independent,

$$CI_W = [\,\hat{p} + z^2/2n \pm z\sqrt{(\,\hat{p}(1-\hat{p})/n + z^2/4n^2\,)}\,] \,/\, (1 + z^2/n) \qquad (6)$$

to show how much too narrow a sample-level interval would be. To test whether the gap is real, we compute for each photograph s the difference d_s between its accuracy under the two protocols and run an exact paired sign-flip test over all 2^n sign assignments:

where:

CI_W: Wilson score interval for a proportion (0–1);

$\hat{p}$: observed accuracy as a proportion (0–1);

n: number of samples (4,194 for soil type, 294 for soil condition);

z: standard normal quantile, 1.96 for a 95% interval.

$$p = \#\{\,\sigma \in \{\pm 1\}^n : |mean(\sigma_s\ d_s)| \geq |mean(d_s)|\,\} \,/\, 2^n \qquad (7)$$

where:

p: two-sided exact p-value (0–1); the smallest value attainable with n photographs is 2^(1−n);

n: number of source photographs (16 for soil type, 14 for soil condition), so 65,536 and 16,384 sign assignments are enumerated;

σ: a vector of signs in $\{-1, +1\}^n$;

d_s: difference in accuracy on photograph s between the two protocols (−1 to +1);

mean(·): arithmetic mean over the n photographs.

McNemar's test [33] on the paired sample-level predictions is reported alongside, although it overstates significance for the same reason as the Wilson interval. We also report per-fold and per-photograph accuracies, the majority-class rate as a baseline, and a photograph-identification test in which the same features and classifier predict which source photograph a tile came from under 5-fold cross-validation. Finally, the EfficientNetV2-B0 evaluation was repeated with five independent seeds for the random crops and augmentation, to measure how much of the result depends on the particular tiles drawn.

3.8 Proposed probe-validity test and its simulation

The firmware of §3.3 cannot detect a stopped pump or a failed probe. We propose, but have not yet implemented in firmware, a test that uses the pump to check the probe. After delivering a known volume V, the controller samples the probe at 10 Hz and measures the onset delay τ of the response, its steepest rise g and its peak per unit volume π:

$$g = max_t\, dM/dt, \quad \pi = (max_t\, M(t) - M_0) / V \qquad (8)$$

where:

M_0: mean moisture over the 5 s before the pulse, % (0–100);

M(t): moisture after the pulse, sampled at 10 Hz for 90 s, % (0–100);

V: delivered volume, mL (> 0); 50 mL in the simulation;

g: steepest rise, percentage points per second (> 0); flagged above g_max = 8.0;

π: peak rise per unit volume, percentage points per mL (> 0); flagged below π_min = 0.239;

τ: onset delay before the moisture begins to rise, s (≥ 0); flagged below τ_min = 3.6.

A probe that shows almost no rise after water was delivered (π below a minimum) is flagged, as is one that jumps immediately and steeply (τ below a minimum and g above a maximum), which indicates free water rather than soil. Because the test looks at the change caused by a known input rather than at the absolute reading, it should still work when the probe's calibration has drifted (Algorithm 2).

```
Algorithm 2: Proposed pulse-response check (not in firmware)
1  M_0 <- mean moisture over 5 s;  deliver volume V;  sample M(t) at 10 Hz for 90 s
2  compute tau, g, pi                                        // Eq. (8)
3  if pi < PI_MIN                      -> FAULT  (no response to delivered water)
4  elif tau < TAU_MIN and g > G_MAX    -> FAULT  (probe in free water)
5  else                                -> VALID  (continue normal control)
6  on FAULT: relay OFF, raise alert, stop acting on the probe reading
```

We checked the decision logic, not the physics, in a Monte Carlo simulation. The probe response was modelled as the difference of two exponentials, with onset delay 3–15 s, rise constant 3–10 s, drainage constant 60–300 s and a peak of 0.30–0.50 percentage points per millilitre for a 50 mL dose. Sensor noise was drawn between 0.50 and 0.94 percentage points, the median and 90th percentile of the standard deviation measured in flat 30-sample windows of our telemetry log, and readings were rounded and clamped as in the firmware. Seven conditions were simulated: healthy; offset drift of up to ±20 points; sensitivity drift to 0.6–0.9 of nominal; fouling (0.05–0.25 of nominal, slower onset); a dislodged probe with no response; a disconnected probe pinned at 0%; and a probe in free water. The three thresholds were set from 3,000 draws of the three normal conditions only (1st percentile of π and τ, 99th of g) and then frozen, so the faults played no part in choosing them. Each condition was then tested on 2,000 fresh draws, and a sweep over probe sensitivity from 0 to 1 located the point at which detection fails. A static check that flags only readings pinned at 0% or 100% served as the comparison.

4. Experimental Results and Discussion

4.1 Experimental conditions

Hardware tests used the prototype of Fig. 3 with the probe in a sealed container of soil that was wetted and allowed to dry. The ESP32 was powered over USB and the relay and pump from a separate 5 V supply. Classifier evaluations ran on the host in TensorFlow and scikit-learn with all seeds fixed; the three backbones were evaluated on one shared set of tiles so that they can be compared directly.

4.2 Hardware tests

Table 5: Hardware functional tests. Each fault was injected once as a functional check; repeatability is discussed below and in §5.

Test	Type	Input	Expected	Observed
TC-01	Normal	Moisture 65%, 28 °C, 60% RH	Pump off; green off; red on; telemetry sent	As expected. Pass
TC-02	Normal	Moisture 22%	Pump on; green LED blinks every 250 ms	As expected. Pass
TC-03	Boundary	Moisture 45% with pump already on	Pump stays on	State held, no chatter. Pass
TC-04	Boundary	Moisture exactly 30%	Pump off (rule is M < 30)	As expected. Pass
TC-05	Fault	DHT11 data line removed during operation	Null values in JSON; irrigation continues	As expected. Pass
TC-06	Fault	Moisture probe unplugged	Reading clamps to 0%; pump turns on	Pump turned on as a fail-safe, but the fault itself was not detected
TC-07	Fault	Pump supply removed while running	Pump stops; ESP32 keeps running	Pump stopped but firmware kept reporting it on. Fail (undetected)

Table 5 records seven fault-injection tests, each run once. Five behaved exactly as specified; the two that did not are both faults the firmware has no way to observe. TC-07, in which the pump stopped while the node went on reporting that irrigation was taking place, is the more consequential and is examined in §4.3.

TC-01 to TC-05 behaved as designed; TC-05 shows the node keeps irrigating when the DHT11 fails. TC-06 is a fail-safe rather than a detection: the node waters the soil because the reading falls to 0%, but it does not know the probe is gone, and a probe that had drifted instead of being unplugged would produce no symptom at all.

Each fault was injected once, so these tests show behaviour but not its consistency. Two aspects of repeatability can nevertheless be taken from the telemetry log. Across 7,949 within-session intervals the median time between records was 1.00 s (interquartile range 1.00–2.00 s, 95th percentile 3.00 s), which bounds how quickly the host sees a change. The hysteresis behaviour was observed over all 33 sessions (§4.4). The log contains no missing temperature or humidity values, so TC-05 could not be cross-checked against logged data. Repeated fault injection with timing measurements is listed as future work in §5.

4.3 Silent actuator failure (TC-07)

TC-07 is the most consequential hardware result. When the pump supply was cut, the pump stopped, but the firmware kept its internal state at “on”, kept the green LED blinking and kept sending “pump”: “ON”. Nothing in the system could tell that no water was flowing. In the field this failure would last until the soil dried enough for someone to notice the crop, and every record in the meantime would say that irrigation was taking place.

The cause is structural: the relay output is commanded but never measured. Two remedies follow. A current sensor on the pump supply, such as an ACS712 module listed at ₹145 [24], would show directly whether the pump draws current. The pulse-response test of §3.8 would also catch it, because a stopped pump delivers no water, the probe shows no rise, and π falls below its threshold. The test cannot say whether the pump or the probe is at fault, but combined with the current sensor it can: current flowing with no probe response points to the probe, and no current points to the pump.

4.4 Control-law behaviour

Fig. 5 shows a drying and rewetting cycle from the log, with the pump state reconstructed from the recorded moisture for two rules: a single set point at 30% and the hysteresis rule of Eq. (2). In this 14-minute session, which begins at 78% moisture, the single set point would have switched the relay six times and the hysteresis rule four. Over all 33 sessions the totals are 69 and 57, a reduction of 17%. The benefit is modest because only 6.6% of logged readings fall inside the 30–60% band; the probe spends most of its time well away from the region where small fluctuations could toggle the relay.

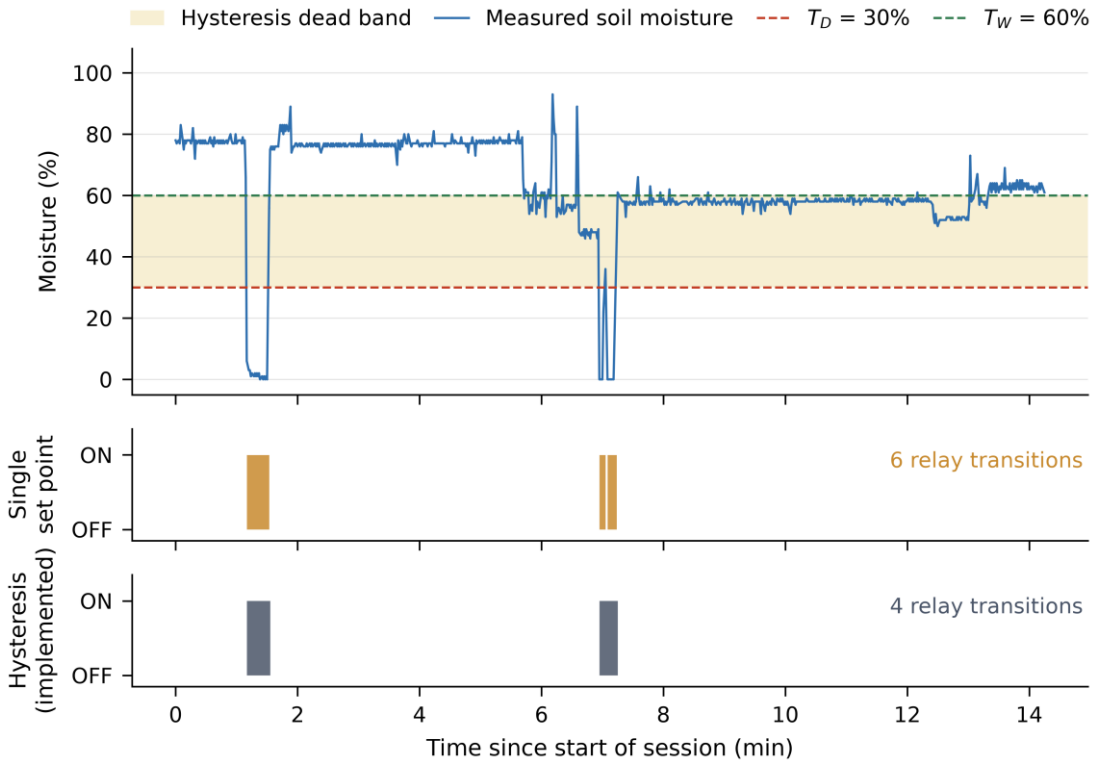


Figure 5: Measured drying and rewetting cycle with pump state reconstructed under a single set point and under the implemented hysteresis rule.

Figure 5 plots a measured 14-minute session, beginning at 78% moisture, with the pump state reconstructed under both rules. A single set point would have switched the relay six times against four for the implemented hysteresis rule. Every logged sample is marked, so the flat stretches are measured values rather than a line drawn across a gap in the log.

4.5 Classification under the two protocols

Table 6 and Fig. 6 give the main result of this study.

Table 6: Accuracy under both protocols, with 95% bootstrap intervals over source photographs and the exact sign-flip test.

Task	Backbone	Augmentation-level (95% CI)	Source-disjoint (95% CI)	Gap Δ (pp)	Sign-flip p
Soil type	EfficientNetV2-B0	94.0 (91.2–96.5)	46.8 (29.7–64.4)	47.2	3.1×10^{-5}
Soil type	MobileNetV2	91.9 (87.8–95.3)	46.8 (29.1–64.9)	45.0	3.1×10^{-5}
Soil type	ResNet50	92.5 (89.3–95.5)	44.6 (27.4–62.7)	47.9	3.1×10^{-5}
Soil condition	EfficientNetV2-B0	99.7 (99.0–100.0)	86.1 (67.7–99.3)	13.6	0.125
Soil condition	MobileNetV2	99.7 (99.0–100.0)	79.9 (60.2–95.6)	19.7	0.031
Soil condition	ResNet50	98.6 (96.9–100.0)	75.5 (57.5–91.2)	23.1	0.0078
Soil type	Majority-class baseline	43.3	43.3	—	—
Soil condition	Majority-class baseline	28.6	28.6	—	—

Table 6 is the central result of the paper. Holding out whole photographs drops soil-type accuracy from 91.9–94.0% to 44.6–46.8%, a gap of 45.0–47.9 points, and soil-condition accuracy from 98.6–99.7% to 75.5–86.1%, a gap of 13.6–23.1 points. The soil-type gap is well above the 5–30 points reported for medical images [13], and the exact test over photographs makes it certain for every backbone ($p = 3.1 \times 10^{-5}$) while leaving the condition gap significant for only two backbones of three.

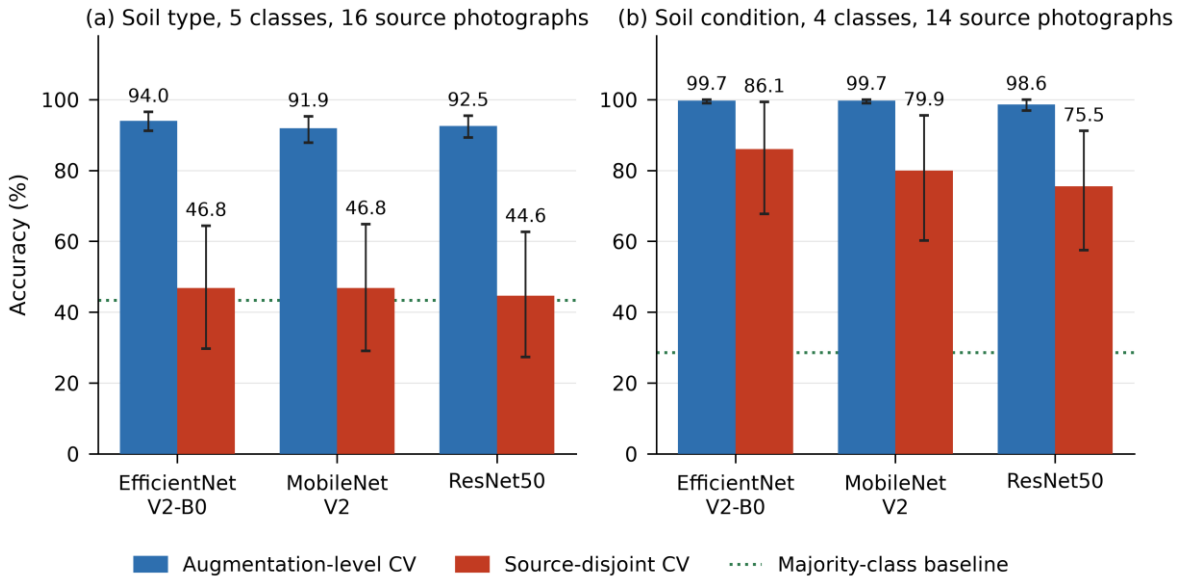


Figure 6: Accuracy under each protocol for three backbones, with 95% bootstrap intervals over source photographs. The dotted line is the majority-class baseline.

Figure 6 shows the result of Table 6 together with its uncertainty. The augmentation-level bars stand above 91% for every backbone and both tasks, while the source-disjoint bars for soil type fall to the dotted majority-class line at 43.3%. The intervals, taken over photographs rather than samples, are wide enough for soil type that the accuracy on its own would mislead.

The source-disjoint soil-type accuracy is close to the majority-class rate of 43.3%, the score a classifier would obtain by always answering “Clay”. In accuracy the classifier barely improves on that rule, although its macro-F1 of 0.233 is about twice the 0.121 the rule would score, so it has learned something about the two best-represented classes. The condition classifiers hold up much better, at 2.6 to 3 times their 28.6% baseline.

The result does not depend on the backbone. EfficientNetV2-B0, MobileNetV2 and ResNet50 differ in design and in feature size, yet their source-disjoint accuracies for soil type lie within 2.2 points of one another, and all three show gaps of at least 45 points. For soil condition the backbone matters more: source-disjoint accuracy ranges from 75.5% with ResNet50 to 86.1% with EfficientNetV2-B0, for reasons examined in §4.8. The classifier is not the cause either: in a separate run with a different tiling draw, a 500-tree random forest on the EfficientNetV2-B0 and colour features fell from 92.5% to 41.9% on soil type, the same collapse as the SVM. Nor does the soil-type result depend on the particular tiles drawn: over five independent tiling seeds, EfficientNetV2-B0 scored 93.0 ± 0.4% augmentation-level and 44.9 ± 1.5% source-disjoint for soil type (gap 48.1 ± 1.3 points, range 46.3–49.7), and 99.4 ± 0.4% against 85.5 ± 1.6% for soil condition (gap 13.9 ± 1.8). Seed-to-seed variation is small beside the gap itself, and the primary run reported in Table 6 lies inside the range of the five seeds.

4.6 Uncertainty and significance

Fig. 7 shows how accuracy is distributed. Under the augmentation-level protocol the five folds agree closely. Under the source-disjoint protocol, accuracy on individual photographs ranges widely: with EfficientNetV2-B0, 2 of the 16 soil-type photographs are never classified correctly and none is classified perfectly, and the standard deviation across photographs is 35.9 points. The overall accuracy is an average over very different photographs, and its uncertainty is correspondingly wide.

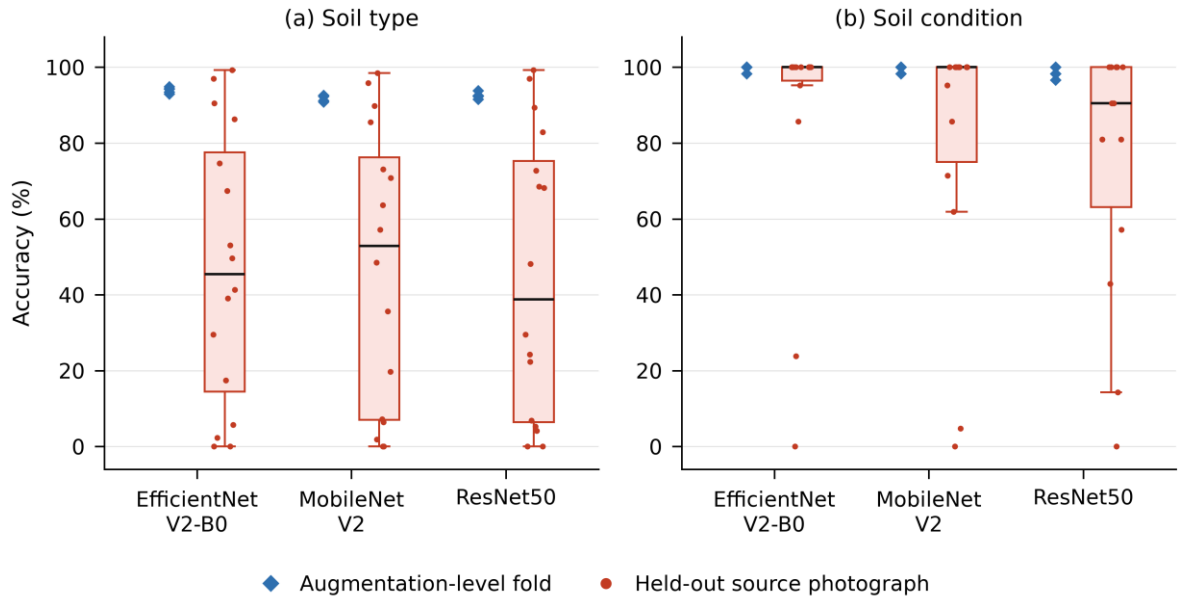


Figure 7: Accuracy per augmentation-level fold (diamonds) and per held-out source photograph (dots and boxes).

Figure 7 explains why those intervals are so wide. The five augmentation-level folds agree to within about two points, whereas accuracy on individual held-out photographs covers almost the whole range from 0 to 100%. With EfficientNetV2-B0, 2 of the 16 soil-type photographs are never classified correctly and none is classified perfectly, giving a standard deviation of 35.9 points.

Table 7 compares the two ways of estimating uncertainty. The Wilson interval, which treats all 4,194 soil-type samples as independent, is about 3 points wide; the bootstrap over photographs is about 35 points wide. McNemar's test on samples declares every gap in Table 6 significant, with p-values between 7.0×10^{-10} and effectively zero. The exact test over photographs is more discriminating. For soil type, all 16 photographs scored lower under the source-disjoint protocol with every backbone, giving $p = 3.1 \times 10^{-5}$, the smallest value possible with 16 photographs. For soil condition, 10 of the 14 photographs were classified perfectly under both protocols with EfficientNetV2-B0, so the gap rests on

4 photographs and p = 0.125. With MobileNetV2 and ResNet50 more photographs are affected (6 and 8) and p = 0.031 and 0.0078. The soil-type gap is therefore firmly established. The soil-condition gap is large on average but is carried by a few photographs, and it cannot be called significant for every backbone, a distinction the sample-level test hides entirely.

Table 7: Sample-level versus photograph-level uncertainty (EfficientNetV2-B0, source-disjoint).

Statistic	Soil type (16 photographs)	Soil condition (14 photographs)
Accuracy	46.8%	86.1%
95% Wilson interval (treats samples as independent)	45.3–48.3%	81.6–89.6%
95% bootstrap over photographs	29.7–64.4%	67.7–99.3%
McNemar p (samples)	$< 10^{-300}$	7.0×10^{-10}
Exact sign-flip p (photographs)	3.1×10^{-5}	0.125
Photographs scoring lower under source-disjoint	16 of 16	4 of 14 (10 perfect under both)

Table 7 is the reason the intervals in Table 6 are quoted over photographs rather than over samples. For the same predictions the two interval estimates differ by an order of magnitude in width, and the two significance tests disagree about the soil-condition gap. Reporting only the sample-level figures would have made a result that rests on four photographs look decisive.

4.7 Why soil photographs leak so badly

The photograph-identification test shows how strongly tiles carry the identity of their source. Given only the soil-type features, the classifier names which of the 16 photographs a tile came from with 91.5% accuracy, against a chance rate of 6.3%; the colour descriptor alone manages 57.0%. The soil-condition features identify their photograph with 99.7% accuracy. Under the augmentation-level protocol, a classifier that recognises the photograph gets the soil label right without learning anything about soil.

The per-class results in Table 8 and Fig. 8 show the second mechanism. Source-disjoint F1 rises with the number of photographs available for a class, and the two classes with a single photograph, Loam and Sandy Loam, score zero with every backbone. This follows from the protocol: holding out the only photograph of a class removes that class from the training set, so the classifier cannot predict it. The confusion matrix in Fig. 9(a) shows those two rows spread across the other classes, with an empty diagonal.

Table 8: Soil-type F1 under source-disjoint validation, by number of source photographs.

Class	Photographs	Samples	F1, EfficientNetV2-B0	F1, MobileNetV2	F1, ResNet50
Clay	7	1818	0.739	0.707	0.697
Sand	5	1320	0.383	0.411	0.352
Loamy sand	2	528	0.042	0.046	0.059
Loam	1	264	0.000	0.000	0.000
Sandy Loam	1	264	0.000	0.000	0.000
Macro average	—	4,194	0.233	0.233	0.222

Table 8 shows source-disjoint F1 rising with the number of photographs behind a class: 0.70–0.74 for Clay with seven, 0.35–0.41 for Sand with five, below 0.06 for Loamy sand with two, and exactly zero for the two classes represented by a single photograph. A class with one photograph cannot be predicted at all once that photograph is held out, which is why the macro average stays near 0.23.

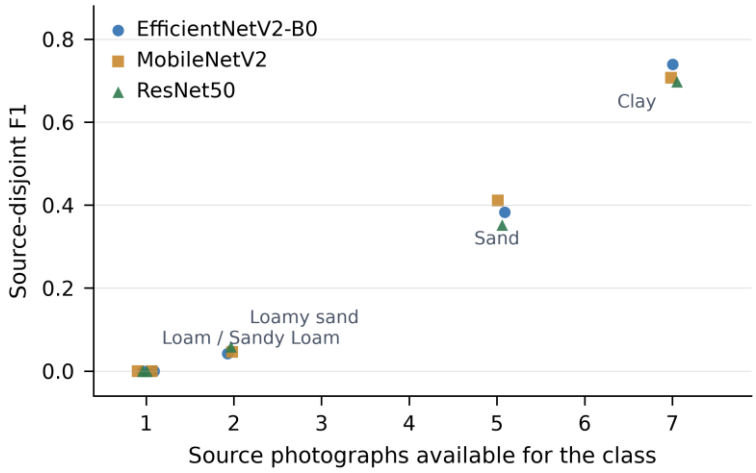


Figure 8: Source-disjoint F1 against the number of source photographs per class, for three backbones.

Figure 8 plots source-disjoint F1 against the number of photographs behind each class, for all three backbones. The relationship is monotone and steep, and the two single-photograph classes sit at zero whichever backbone is used. The practical reading is that more source photographs, not a different network, is what would lift this curve.

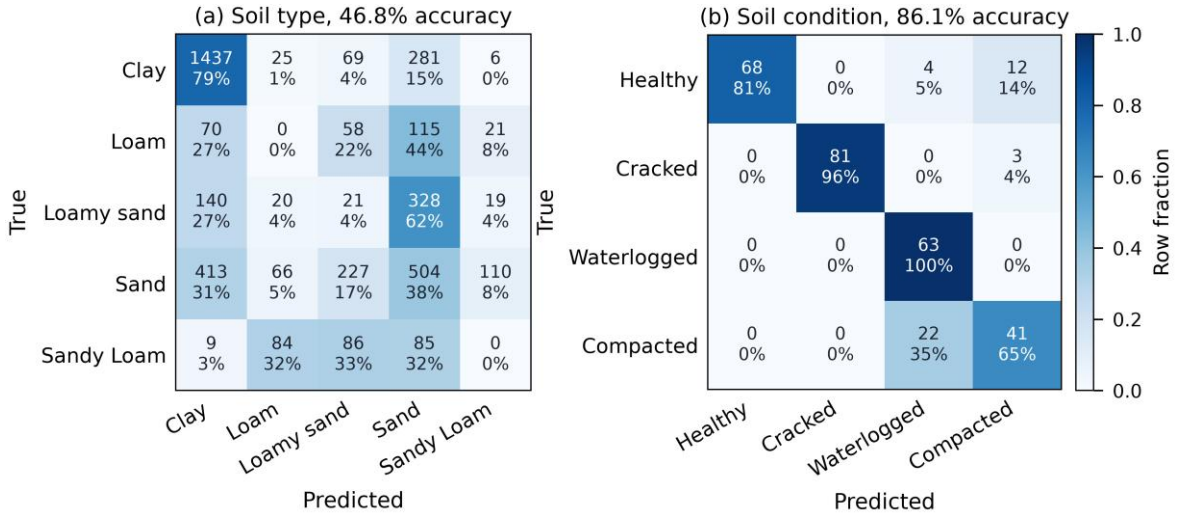


Figure 9: Source-disjoint confusion matrices for EfficientNetV2-B0: (a) soil type, (b) soil condition. Counts and row percentages.

Figure 9 shows where the errors go. For soil type the Loam and Sandy Loam rows have an empty diagonal and spread across the other classes, and 413 of the 1,320 Sand samples are read as Clay. For soil condition the only substantial error is the 22 of 63 compacted samples read as waterlogged, which is what lowers waterlogged precision to 0.708.

These three properties, texture images cut into many tiles, one photograph per field sample with few samples per class, and a shared photographic signature across tiles, are why soil photographs differ from the leaf images for which post-split augmentation is enough [18], and they may also explain why the gap here exceeds the range reported for medical imaging [13].

4.8 Soil condition, and whether the errors are architectural

For soil condition the errors are concentrated in very few images. With EfficientNetV2-B0, 37 of the 41 misclassified samples come from two of the 14 source images. All 21 samples of one compacted image are read as waterlogged; unlike the other two compacted images, which show dry clods, this one

shows a smooth, grey surface with a wet sheen that resembles the waterlogged class. Sixteen of the 21 samples of one healthy image are misread; it is the only very dark healthy image, with visible plant debris. The 22 compacted and 4 healthy samples read as waterlogged are what lower waterlogged precision to 0.708. Cracked soil, a large-scale pattern that survives a change of image, is recognised most reliably.

An obvious question is whether this confusion reflects the data or the model. The same compacted image is scored 0% by all three backbones, and compacted-to-waterlogged confusion accounts for 22, 21, 26 errors respectively (Table 9). A failure that repeats on the same image across three networks of different design points to the image rather than the architecture; with three compacted images, one atypical example is enough to produce it. The backbones do differ on healthy soil: MobileNetV2 and ResNet50 read 23 and 17 healthy samples as waterlogged, against 4 for EfficientNetV2-B0, which is why their condition accuracy is lower. The evidence is consistent with a limitation of the data, though it does not rule out that a model trained on many more images would separate the classes.

Table 9: Soil-condition F1 under source-disjoint validation for three backbones.

Class	Photographs	EfficientNetV2-B0	MobileNetV2	ResNet50
Healthy	4	0.895	0.828	0.830
Cracked	4	0.982	0.909	0.859
Waterlogged	3	0.829	0.728	0.737
Compacted	3	0.689	0.724	0.562
Macro F1	—	0.849	0.797	0.747
Compacted read as waterlogged (of 63)	—	22	21	26

Table 9 gives per-class soil-condition F1 for the three backbones. Cracked soil, whose pattern survives a change of image, is recognised most reliably at 0.859–0.982; compacted soil, which has three source images of which one is atypical, is recognised least reliably at 0.562–0.724. The ordering of the four classes is the same for all three backbones, which is what makes the images rather than the architecture the likely explanation.

4.9 Simulated pulse-response test

The three thresholds were set from normal behaviour alone — π_{min} = 0.239 points per mL, τ_{min} = 3.6 s and g_{max} = 8.0 points per second — and then frozen. Table 10 and Fig. 10 give the outcome. The offset-drift result is the important one: a probe reading 20 points too high or too low passes, correctly, because it still responds to water.

Table 10: Simulated flag rates (n = 2000 draws per condition).

Condition	Should be flagged?	Pulse-response test	Static clamp check
Healthy	No	0.0%	0.0%
Offset drift (±20 points)	No	0.0%	0.0%
Sensitivity drift (0.6–0.9×)	No	2.2%	0.0%
Fouled (0.05–0.25×)	Yes	100.0%	0.0%
Dislodged, no response	Yes	100.0%	0.0%
Disconnected, pinned at 0%	Yes	100.0%	100.0%
Probe in free water	Yes	100.0%	0.0%

Table 10 reports the simulated flag rates. With thresholds fixed from normal behaviour alone, the pulse-response test flags none of the healthy or offset-drifted probes and 2.2% of the gain-drifted ones, while

flagging every fouled, dislodged, disconnected and submerged probe. The static clamp check catches only the disconnected probe, and would raise the same flag for genuinely air-dry soil.

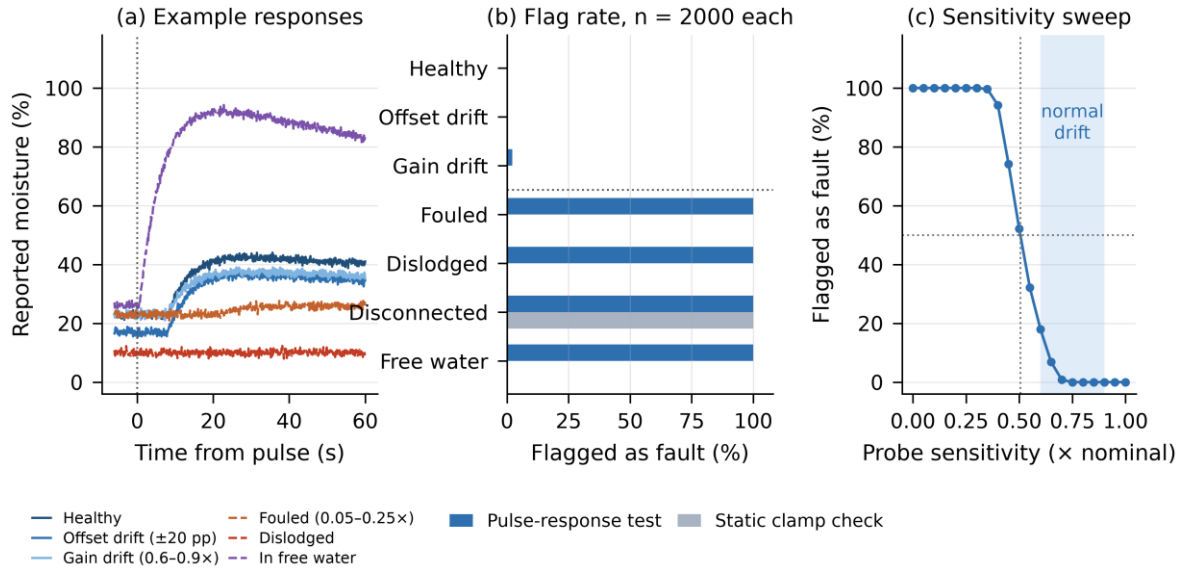


Figure 10: Simulation of the proposed test: (a) example responses, (b) flag rates, (c) flag rate against probe sensitivity.

Figure 10 shows the three things the simulation has to establish. Panel (a) is the response model the decision rules act on, panel (b) the flag rates of Table 10, and panel (c) the sensitivity sweep discussed below. Reading (a) together with (b), the fouled and dislodged traces are caught because they never rise after the pulse, and the free-water trace because it rises at once.

The sweep in Fig. 10(c) shows where the test stops working. Detection falls to 50% when the probe responds at 0.51 of its nominal sensitivity, and a probe that has lost less than about 30% of its sensitivity is almost never flagged. Moderate degradation therefore passes as normal drift; that is the price of keeping false alarms near 2%. These results show that the decision rules separate the modelled faults; they do not show that real soil and real probes behave like the model. That requires the fault-injection experiment described in §5.

4.10 Fault-handling summary

Table 11: Fault handling in the implemented system and under the proposed test.

Fault	Current firmware	Consequence	With proposed test (simulated)
DHT11 failure	Detected by isnan()	Irrigation continues on moisture alone	Unchanged
Probe disconnected	Not detected; reading clamps to 0%	Pump runs as a fail-safe	Flagged (100%)
Probe dislodged or fouled	Not detected	Wrong reading acted on	Flagged (100%)
Probe calibration drift	Not detected	Wrong reading acted on	Passes correctly; responds to water
Pump power lost (TC-07)	Not detected	System reports irrigation that is not happening	Flagged as no response; ACS712 would identify the pump
Serial link lost	Host timeout	Dashboard shows no data; ESP32 keeps irrigating	Unchanged

Table 11 summarises what the node can and cannot detect today. Only the DHT11 failure and the loss of the serial link are handled; probe faults and the stopped pump pass unnoticed. In simulation the proposed test would flag four of these conditions, and would correctly let calibration drift through, because a drifted probe still responds to water.

4.11 Comparison with the baseline system

Table 12: Comparison with Morchid et al. [1].

Aspect	Morchid et al.	This work	Assessment
Sensing	Temperature, humidity, soil moisture, water level	Temperature, humidity, soil moisture, soil photographs	Adds an image channel; no water-level sensor
Control	Fixed threshold, open loop	Hysteresis 30–60%, open loop	17% fewer relay transitions; still open loop
Deployment	Cloud required	Local; control runs without a network	Suits intermittent connectivity
Machine learning	None	Soil condition 75.5–86.1%, soil type 44.6–46.8% (source-disjoint)	Condition promising but needs field data; type not usable
Fault handling	Not reported	DHT11 and serial-link faults detected; TC-06 and TC-07 not	Partial
Cost	≈ US$44	₹970 (≈ US$11–12)	About a quarter of the cost

Table 12 compares the node with the baseline system [1]. It costs about a quarter as much, keeps irrigating without a network and adds an image channel, at the price of no water-level sensor and a control loop that is still open. The machine-learning row is the honest one: soil condition is promising at 75.5–86.1%, but soil type at 44.6–46.8% is not usable.

4.12 Expected versus actual results

Table 13 compares measured results with expectations taken from the firmware, from published figures or from the baseline system. Rows for which we have no independent measurement, such as sensor accuracy against a reference instrument, are left out.

Table 13: Expected versus actual results.

Parameter	Expected (source)	Actual	Comment
Telemetry interval	≈ 1 s (firmware delay)	Median 1.00 s, IQR 1.00–2.00 s	7,949 logged intervals
Pump switch-on	Below 30% (Eq. 2)	28–29% in serial output	Integer comparison, 1 Hz sampling
Dry-air reading	ADC 2590 → 0%	ADC 2559–2590 → 0–2%	ADC noise of about 1–2 points
Relay transitions	Fewer than a single set point (by design)	57 vs 69 over 33 sessions	17% fewer
Soil type accuracy	≈ 91–97% in published work [8], [9]	91.9–94.0% aug.-level; 44.6–46.8% source-disjoint	Only the leaky figure matches the literature
Accuracy inflation from leakage	5–30 points in medical imaging [13]	45.0–47.9 (type), 13.6–23.1 (condition)	Soil type exceeds the published range
Cost	≈ US$44 (baseline [1])	₹970 (≈ US$11–12)	About a quarter

Table 13 checks measured behaviour against what the firmware, the literature or the baseline predict. Telemetry interval, switching threshold, dry-air reading and relay-transition count all behave as designed. The two rows that depart from expectation are soil-type accuracy, where only the

augmentation-level figure resembles published work, and the size of the leakage effect, which exceeds the 5–30 points reported for medical imaging [13].

4.13 Discussion

The implication for the literature is limited but clear. Published soil-classification accuracies of 91–97% [8], [9] were obtained on different data, and we have not re-evaluated them, so we cannot say they are inflated. We can say that without a stated photograph-level split they cannot be read as estimates of performance on new fields, and that on our data the difference is large enough to change the conclusion. Given how widespread leakage is in applied machine learning [12], the split protocol and the number of source photographs per class should be reported as routinely as the accuracy itself.

The practical consequence is easy to state. Someone choosing a soil classifier for an irrigation controller on the basis of its reported accuracy would, in this case, have expected about 94% and received a model close to its majority-class baseline on photographs it had not seen. In a controller, that error becomes a watering decision. The condition classifier performs far better under the strict protocol, although it rests on 14 web-sourced images and should be confirmed on field photographs. For both tasks the most direct route to a usable classifier is more source photographs rather than a different network.

A separate point concerns the multimodal fusion stage that was part of the original design, in which sensor readings are appended to the image features. It is not evaluated here because no dataset pairs soil photographs with probe readings taken at the same time and place.

In the deployed code, three normalised sensor values are appended to the image features at inference time. The saved classifier was trained on image features only, so the inference code trims the vector back to its training length and the sensor values are dropped. Every prediction from the deployed system is therefore image-only, which is also what is evaluated in this paper.

5. Limitations and Future Work

The main limitation is the number of source photographs: 16 for soil type and 14 for soil condition, with two soil-type classes represented by a single photograph. The wide intervals in Table 7 follow directly from this, and source-disjoint results for single-photograph classes are zero by construction. Collecting several hundred field photographs, balanced across classes and taken under different light and with different devices, is the most useful next step. Until then the soil-type classifier should not drive irrigation decisions.

The soil-condition images come from a web image search rather than from our own fields, their provenance was not recorded, and they may include staged or edited photographs. The condition results should therefore be read as a test of the evaluation protocol on this collection, not as evidence of field performance, and a field-collected set with known provenance is needed before the condition classifier is deployed.

The measurement also rests on one soil-type dataset. Repeating it on a larger public collection, such as the 1,269-image set used in [9], would show whether the size of the gap is a property of soil photographs generally or of this collection.

Each hardware fault was injected once. Repeating every fault test at least five times, with the relay response and the time to reach the fail-safe state measured on an oscilloscope or logic analyser, is needed before the node’s behaviour can be called consistent rather than merely correct.

The pulse-response test exists only in simulation, and the simulation checks the decision rules, not the soil physics. Implementing it needs a metered dose (a flow sensor, or pump run-time at a measured flow

rate) and a 10 Hz sampling burst, followed by an experiment in which real probes are fouled, partly withdrawn or offset and the detection and false-alarm rates are measured. Adding an ACS712 current sensor would close the TC-07 gap independently.

Finally, all hardware work took place in a container rather than a field, and the 4.5 hours of logged operation say nothing about long-term durability. The telemetry log should also record raw ADC values so that clamped readings can be diagnosed afterwards.

6. Conclusion

We evaluated the soil classifiers of a low-cost edge irrigation node twice on identical features, once with the usual augmentation-level split and once with each source photograph held out in turn. Soil-type accuracy fell from 91.9–94.0% to 44.6–46.8%, close to the majority-class rate, and soil-condition accuracy fell from 98.6–99.7% to 75.5–86.1%. For soil type the gap held across three backbones and five tiling seeds and was significant for every backbone under an exact test over photographs; for soil condition it was smaller, rested on a few images and was significant for two backbones of three.

Two properties of soil photographs account for it. Tiles carry the identity of their photograph so strongly that a classifier can name the photograph from a tile, and classes with only one photograph cannot be learned at all once that photograph is held out. The reported accuracy of a soil classifier trained on few photographs therefore says more about the dataset than about the network.

On the hardware side, the ₹970 node runs its control loop without a network, keeps irrigating when its air sensor fails, and switches its relay 17% less often than a single set point would. It cannot detect a stopped pump. A pulse-response test, checked in simulation, would flag that fault together with fouled, dislodged, disconnected and submerged probes, while passing probes that are merely miscalibrated; confirming this on real hardware is the next step.

We recommend that soil-image accuracy be reported with a stated photograph-level split, the number of source photographs per class, a majority-class baseline and an interval estimated over photographs.

CRediT authorship contribution statement

Utkarsh Persai: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft. Yash Chaubey: Data curation, Validation, Visualization, Writing – review and editing. Venkata Hrishi Samay Jetty: Software, Validation, Investigation, Writing – review and editing. Devansh Rathore: Hardware, Validation, Investigation, Resources. Syamasudha Veeragandham: Supervision, Project administration, Conceptualization, Writing – review and editing.

Funding Statement

The authors declare that no funding, grants, or external support were received during the development of this work.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The IRSID soil-type images are available from IEEE DataPort [25]. The soil-condition images were obtained through a web image search and are not redistributed, because their licences could not be confirmed; the script that collected them is included instead. The telemetry log, the ESP32 firmware, the evaluation script (training/source_disjoint_eval.py) that produces every accuracy, interval and test in §4.5–4.8, and the simulation script (training/simulate_probe_check.py) are available at https://github.com/utki445/smart-agriculture-ai. Weather forecasts were obtained from the Open-Meteo API [23].

References

[1] A. Morchid, H. Qjidaa, R. El Alami, S. Mobayen, P. Skruch, and B. Bossoufi, "Smart irrigation-based internet of things and cloud computing technologies for sustainable farming," Scientific Reports, vol. 16, art. 5293, 2026. doi:10.1038/s41598-026-35810-0

[2] A. A. Abdelmoneim, H. N. Kimaita, C. M. Al Kalaany, B. Derardja, G. Dragonetti, and R. Khadra, "IoT sensing for advanced irrigation management: a systematic review of trends, challenges, and future prospects," Sensors, vol. 25, no. 7, art. 2291, 2025. doi:10.3390/s25072291

[3] M. Pramanik, M. Khanna, M. Singh, D. K. Singh, S. Sudhishri, A. Bhatia, and R. Ranjan, "Automation of soil moisture sensor-based basin irrigation system," Smart Agricultural Technology, vol. 2, art. 100032, 2022. doi:10.1016/j.atech.2021.100032

[4] R. Sreeram, S. A. Krishna, A. S. Kumar, S. Remya, and Y. Y. Cho, "Soil moisture monitoring technologies in smart agriculture: a comprehensive review," Farming System, vol. 4, no. 2, art. 100189, 2026. doi:10.1016/j.farsys.2025.100189

[5] D. A. Aranda Britez, A. Tapia, and P. Millán Gata, "A self-calibration algorithm for soil moisture sensors using deep learning," Applied Intelligence, vol. 55, no. 4, art. 276, 2025. doi:10.1007/s10489-024-05921-0

[6] R. Pahuja, "Development of semi-automatic recalibration system and curve-fit models for smart soil moisture sensor," Measurement, vol. 203, art. 111907, 2022. doi:10.1016/j.measurement.2022.111907

[7] C. K. Gasch, D. J. Brown, E. S. Brooks, M. Yourek, M. Poggio, D. R. Cobos, and C. S. Campbell, "A pragmatic, automated approach for retroactive calibration of soil moisture sensors using a two-step, soil-specific correction," Computers and Electronics in Agriculture, vol. 137, pp. 29–40, 2017. doi:10.1016/j.compag.2017.03.018

[8] R. K. Banoth and B. V. Ramana Murthy, "Soil image classification using transfer learning approach: MobileNetV2 with CNN," SN Computer Science, vol. 5, no. 1, art. 199, 2024. doi:10.1007/s42979-023-02500-x

[9] S. Jalapur and N. Patil, "An integrated deep learning framework for soil type classification," SN Computer Science, vol. 6, no. 3, art. 251, 2025. doi:10.1007/s42979-025-03759-y

[10] V. Pandey, R. Das, and D. Biswas, "AgroSense: an integrated deep learning system for crop recommendation via soil image analysis and nutrient profiling," arXiv:2509.01344, 2025.

[11] A. Katharria, M. Pant, J. D. Velásquez, V. Snášel, K. Rajwar, and K. Deep, "Information fusion in smart agriculture: machine learning applications and future research directions," Information Fusion, vol. 129, art. 104040, 2026. doi:10.1016/j.inffus.2025.104040

[12] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, art. 100804, 2023. doi:10.1016/j.patter.2023.100804

[13] I. E. Tampu, A. Eklund, and N. Haj-Hosseini, "Inflation of test accuracy due to data leakage in deep learning-based classification of OCT images," Scientific Data, vol. 9, art. 580, 2022. doi:10.1038/s41597-022-01618-6

[14] N. Bussola, A. Marcolini, V. Maggio, G. Jurman, and C. Furlanello, "AI slipping on tiles: data leakage in digital pathology," in Pattern Recognition. ICPR International Workshops and Challenges, Lecture Notes in Computer Science, vol. 12661. Cham: Springer, 2021, pp. 167–182. doi:10.1007/978-3-030-68763-2_13

[15] D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, et al., "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," Ecography, vol. 40, no. 8, pp. 913–929, 2017. doi:10.1111/ecog.02881

[16] K. John, D. D. Saurette, and B. Heung, "The problematic case of data leakage: a case for leave-profile-out cross-validation in 3-dimensional digital soil mapping," Geoderma, vol. 455, art. 117223, 2025. doi:10.1016/j.geoderma.2025.117223

[17] F. Chen and J. Vanderborght, “Matching data splitting to extrapolation targets in soil machine learning: when is soil-wise validation needed?,” Environmental Science & Technology, vol. 60, no. 31, pp. 21391–21393, 2026. doi:10.1021/acs.est.6c06032

[18] M. H. Ashmafee, T. Ahmed, S. Ahmed, M. B. Hasan, M. N. Jahan, and A. B. M. A. Rahman, “An efficient transfer learning-based approach for apple leaf disease classification,” arXiv:2304.06520, 2023.

[19] Z. Qin, C. Wang, X. Zhou, T. Yao, W. Yin, H. Liang, and J. Ou, “A transfer learning and data augmentation approach for classifying field images of granite residual slope soils,” Algorithms, vol. 19, no. 9, art. 755, 2026. doi:10.3390/a19090755

[20] M. U. Tariq, S. M. Saqib, T. Mazhar, M. A. Khan, T. Shahzad, and H. Hamam, “Edge-enabled smart agriculture framework: integrating IoT, lightweight deep learning, and agentic AI for context-aware farming,” Results in Engineering, vol. 28, art. 107342, 2025. doi:10.1016/j.rineng.2025.107342

[21] Y. Yang, J. Hu, D. O. Porter, T. H. Marek, C. C. Hillyer, and L. Sun, “Agricultural sensor placement and fault detection in wireless sensor networks,” U.S. Patent 11,520,647 B2, Dec. 6, 2022.

[22] Y. K. Adimoolam, C. Poullis, and M. Averkiou, “Data leakage detection and de-duplication in large scale geospatial image datasets,” arXiv:2304.02296, 2023.

[23] Open-Meteo, “Free weather forecast API,” https://open-meteo.com/ (accessed Sep. 2026).

[24] Robokits India, product listings RKI-1454, RKI-3225, RKI-3154, RKI-3151 and RKI-1838, https://robokits.co.in (accessed 19 Sep. 2026).

[25] D. N. K. Pandiri, R. Murugan, and T. Goel, “Indian Regions Soil Image Database (IRSID): a dataset for classification of Indian soil types,” IEEE DataPort, 2021. doi:10.21227/2zz3-f173

[26] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: a large-scale hierarchical image database,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[27] M. Tan and Q. V. Le, “EfficientNetV2: smaller models and faster training,” in Proc. 38th Int. Conf. Machine Learning (ICML), PMLR vol. 139, 2021, pp. 10096–10106.

[28] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: inverted residuals and linear bottlenecks,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510–4520. doi:10.1109/CVPR.2018.00474

[29] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778. doi:10.1109/CVPR.2016.90

[30] C. Cortes and V. Vapnik, “Support-vector networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[31] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York: Chapman & Hall, 1993.

[32] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927. doi:10.1080/01621459.1927.10502953

[33] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” Psychometrika, vol. 12, no. 2, pp. 153–157, 1947. doi:10.1007/BF02295996